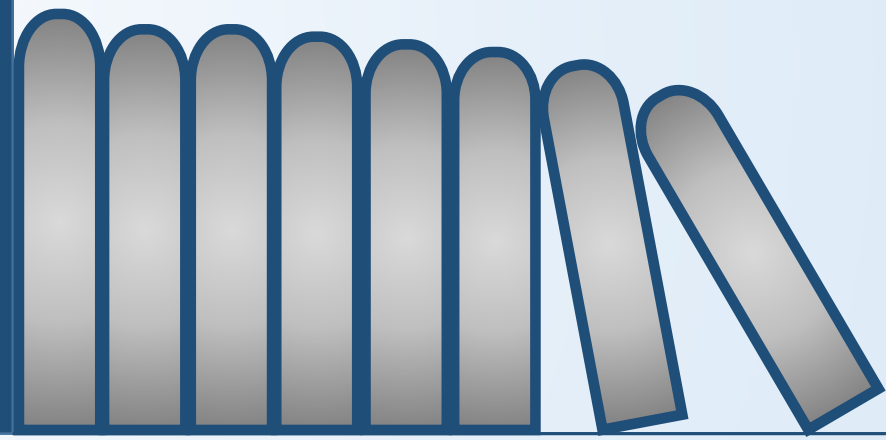
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

哈希长度扩展攻击与碰撞攻击



2026年10月27日星期二



实验目的

- 掌握 MD5 的填充规则与 **Merkle–Damgård** 链式压缩结构
- 理解长度扩展攻击的原理
- 实现长度扩展攻击与 MD5 碰撞的利用
- 掌握防御两类攻击的安全策略选型



本次实验的背景

一、长度扩展攻击

➢ 银行有一个内部 API，用 **MD5(password || message)** 校验请求 —— 密码 **8 个字符**，只有服务端知道

– 你手上有一条 alice 的合法 URL：

http://bank.cse127.local/api?token=...&user=alice&command1=ListSquirrels&command2=NoOp

我们的任务：追加 **&command3=UnlockAllSafes** 让服务端放行；

二、碰撞攻击

➢ 下载站只显示 **MD5** 校验值 —— 没有签名，没有 HTTPS 之外的保证； 本地服务端 api_common.py 是黑盒：只用 token、消息、密码长度

– 不要读源码里的 SECRET；读了密码再伪造，不是长度扩展攻击

我们的任务：用一对碰撞块做出两个 **MD5 相同、行为不同** 的程序



实验内容

四个任务，**25 处填空**，约 30 行代码 —— 从 MD5 结构 → 攻击实战 → 碰撞利用 → 正确防御。

任务	内容	文件
任务一	MD5 的填充与续写（6 空）	`task1_md5.py`
任务二	**长度扩展攻击【★ 主交付物】（8 空）**	`task2_extend.py`
任务三	碰撞与同哈希程序（7 空）	`task3_collision.py`
任务四	HMAC 防御（4 空）	`task4_defense.py`

跑通全链与核对：`python run_all.py` / `python verify.py` —— 交付物核对 **通过 N 项，失败 0 项** 才能交报告



实验原理：哈希函数与消息认证

➢ 哈希把任意长度输入压成固定长度摘要：`H: {0,1}* → {0,1}^n`　（MD5 输出 128 位）

性质	含义	被破坏的后果
单向性	给定摘要，难以找回原消息	口令、承诺值泄露
抗第二原像	给定 m，难以找到 m′≠m 使 H(m′)=H(m)	文件被替换
抗碰撞	难以找到任意一对 m≠m′ 使 H(m)=H(m′)	证书、软件分发被替换

➢ **哈希不是加密：**没有密钥，不能「解密」；需要「带密钥的完整性」时要用 MAC（如 HMAC），输出叫 摘要

本实验要打的构造：**tag = H(secret || message)**　—— 它会被长度扩展攻击



实验原理：Merkle–Damgård 结构与填充

```
          M1 ──▸  M2 ──▸ M3              (每块 64 字节，逐块送入压缩函数 f)
IV ──▸ [f] ──▸ [f] ──▸ [f] ──▸ … ──▸ 摘要 = 处理完最后一块的内部状态
```

摘要就是内部状态 —— 只要知道这个状态，就能**从中间继续算**。这是两种攻击共同的根源。

➢ **MD5 填充规则：** `0x80 | 若干 0x00 | 消息比特数（8 字节，小端序）`

– 零的个数 `zero = (56 - (n+1) % 64) % 64`，填充后总长是 64 的倍数；长度单位是**比特**，要乘 8

消息长度	填充长度	说明
55 字节	9 字节	0x80 + 8 字节长度，不加零
56 字节	72 字节	多出整整一块
64 字节	64 字节	正好一块，仍要再补一整块

填充总是至少加 1 字节；55 / 56 是最常见的边界错误。



实验原理：长度扩展攻击

➢ **服务端的认证规则：** `tag = H(secret ‖ message)`

➢ 攻击者知道 `tag、message、len(secret)`，不知道 secret 的内容 —— 这就够了

– MD5 处理完 `secret‖message‖padding` 后，内部状态正好等于 tag；把 tag 当初始状态继续压 suffix 即可

```
new_tag = MD5_from_state(tag, suffix)
        = MD5(secret ‖ message ‖ padding ‖ suffix)      ← 全程不需要 secret
```

➢ **交给服务端的消息必须是** `message ‖ padding ‖ suffix`（含 0x80/0x00，URL 里要百分号编码）

– 填充长度由 `len(secret)+len(message)` 决定，所以只需要知道密码的**长度**

已知与目标	数值	说明
len(secret)=8, len(msg)=47	secret||message 共 55 字节	填充 9 字节（0x80+8 字节长度）
新消息 47+9+24 = 80 字节	count = (8+47+9)×8 = 512 比特	正好一块，续算从这里起步

不知道密码长度？枚举 0~64 即可 —— **「密码长度保密」挡不住长度扩展**



实验原理：MD5 碰撞

➢ 2004 年**王小云**等人公布 MD5 碰撞。本实验使用一对 **128 字节**的碰撞块：

– 两个文件只有 **6 个字节**不同，差异一共 **6 个比特**；MD5 完全相同，SHA-256 完全不同

偏移	A	B	异或
19	0x87	0x07	0x80
45	0x87	0x07	0x80
59	0x87	0x07	0x80
83	0x87	0x07	0x80
109	0x87	0x07	0x80
123	0x87	0x07	0x80

```
MD5(A) = MD5(B) = 79054025255fb1a26e4bc422aef54eb4    SHA-256(A) ≠ SHA-256(B)
```

碰撞是「起始状态 + 消息」的性质：相同前缀 + 碰撞对 → 通常不再是碰撞；碰撞对 + 相同后缀 → 仍然是碰撞



实验原理：HMAC 与正确的防御

```
HMAC(K, m) = H((K ⊕ opad) ‖ H((K ⊕ ipad) ‖ m))      ipad=0x36, opad=0x5c
```

➢ **为什么抗长度扩展：**tag 是外层哈希的结果，不是任何可续写的中间状态；攻击者不知道 K，构不出外层输入

➢ 校验 tag 用 `hmac.compare_digest`，不要用 ==（逐字节比较会通过计时泄露前缀）

用途	只换 SHA-256 够吗
抗碰撞的文件校验	够（目前）
H(secret‖m) 做 MAC	不够 —— 长度扩展仍然成立
口令存储	不够 —— 要慢哈希 + 盐

长度扩展不依赖 MD5 的具体缺陷 —— **SHA-256 也是 Merkle–Damgård**，换哈希解决不了拼接顺序问题。需要 MAC 时用标准库 `hmac`，不要自己发明构造



实验步骤：填空机制

```
$ python run_all.py --progress
  task1_md5.py  6/6   task2_extend.py  8/8   task3_collision.py  7/7   task4_defense.py  4/4
```

➢ **规矩一：**把 `FILL[n]` 这几个字符删掉，在原位写代码 —— 不要给它赋值

– `FILL[4] = …` ✗ 错，抛 TypeError；`state = …` ✓ 对

➢ **规矩二：**一处填空 = 一个完整表达式或一条完整语句，不要嵌进更大的表达式

➢ **规矩三：**填空只在函数体里 —— 文件可以随时 import、可以 --help

情况	退出码	表现
空还没填	3	FILL[n] not filled yet + 文件名 + 行号（正常状态）
填错了	1	自检里的具体失败项
全对	0	全部通过



实验步骤　任务一：填充与续写

```
$ python task1_md5.py        # 6 处填空：FILL[1] ~ FILL[6]
```

FILL[1]~[3]　padding() 三件套：`bytes([0x80])`、零的个数 `(56-(n+1)%64)%64`、`struct.pack("<Q", n*8)`

FILL[4]~[6]　extend_digest()：旧摘要还原状态 `bytes.fromhex`、count 乘 8 加填充、`extended == direct`

– 续写只需要 `H(original)` 和 `len(original)`，不需要原消息的内容

坑	症状	唯一防线
写成字符串 "0x80"	填充开头不是 80	bytes([0x80])
公式少了 +1	55 字节填充 8 或 10 字节	(56-(n+1)%64)%64
count 漏乘 8 / 漏加填充	续写摘要对不上	count = (len+pad)×8

该看到：55 字节 → 填充 9 字节；56 字节 → 填充 72 字节；续写摘要 = 直接摘要，两者相同：`True`



实验步骤　任务二：长度扩展攻击【★ 主交付物】

```
$ python task2_extend.py     # 8 处填空：FILL[1] ~ FILL[8]
```

FILL[1]~[3]　拆 URL：`query.split("&", 1)`、去掉 `token=` 前缀、`unquote_to_bytes` 只解码一次

FILL[4]~[8]　伪造：填充按 `secret_len + len(msg)` 算；`msg + pad + suffix`；count 是**比特数**；`h.update(suffix)`

– 拼回 URL 用 `quote_from_bytes(new_msg, safe="")`，让 &、=、% 都按字节编码

现象	原因
status = 403	填充长度没加密码长度，或 count 没乘 8
unlock = False	suffix 没有同时进 new_msg 和 h.update()
URL 解析后长度不对	quote_from_bytes 的 safe 不是空字符串

记住：suffix 要出现在**两处** —— 拼进新消息，也要喂给哈希



实验步骤　任务二的输出与解读

```
原始请求:   status = 200   unlock = False
伪造请求:   status = 200   unlock = True    command3 = UnlockAllSafes
填充长度: 9 字节     新消息长度: 80 字节
★ 成功。服务端认为这条请求是 alice 发的。
```

47 字节原消息 + 9 字节填充 + 24 字节 suffix = 80 字节新消息

count = (8 + 47 + 9) × 8 = **512 比特** —— 原消息加密码 55 字节，填充 9 字节，正好一块

– 伪造 URL 已由程序写入 `data/forged_url.txt`，报告里要整条贴出来

服务端没有泄露密码。

它只是相信「token 对得上 = 消息没被改过」这个错误前提 —— 出问题的是把密码拼在消息前面再哈希的用法



实验步骤　任务三：碰撞与同哈希程序

```
$ python task3_collision.py        # 7 处填空：FILL[1] ~ FILL[7]
$ python ../data/good     $ python ../data/evil
```

FILL[1]　找差异：逐字节比较，记录（偏移，异或值，变了几位）

FILL[2]~[3]　攻击 zipapp：`z.writestr(info, PAYLOAD_CODE)`、`buf.getvalue()`（时间戳固定，每次构建逐字节一致）

FILL[4]~[6]　接碰撞块：`good = a + payload`、`evil = b + payload`、`subprocess.run(..., capture_output=True)`

FILL[7]　验证：`md5(good).digest() == md5(evil).digest()`（两边都用 digest）

程序读自己文件的第 `0x13` 字节来决定行为：A 在这个位置是 0x87，B 是 0x07

两段 zipapp 代码**逐字节相同**，差别只在文件开头的碰撞块里 —— 这就是「同 MD5、不同行为」的全部秘密



实验步骤　碰撞的性质与两个程序

```
MD5     A = 79054025255fb1a26e4bc422aef54eb4    相同
SHA-256 A = 8d12236e…  B = b9fef2a8…             不同
加共同前缀 (34 字节) : MD5 还相同?  False
加共同后缀 (22 字节) : MD5 还相同?  True
good 输出: I mean no harm.    evil 输出: You are doomed!
```

操作	MD5 还相同吗
相同前缀 + 碰撞对	否（前缀不同 → 进入 A/B 的起始状态不同）
碰撞对 + 相同后缀	是（相同状态出发压缩同样的后缀）

只校验 MD5 的下载站会把 evil 当成 good。

MD5 已经不适合做文件完整性校验；需要抗碰撞时用 SHA-256 或 SHA-3



实验步骤　任务四：HMAC 防御

```
$ python task4_defense.py        # 4 处填空：FILL[1] ~ FILL[4]
```

FILL[1]　接把 secret 和 msg 拼起来，调库算 SHA-256

FILL[2]　标准库 hmac，算法用 hashlib.sha256

FILL[3]　重新算一遍 HMAC，和 tag 做恒定时间比较

FILL[4]　校验发现能够防御，错误的通不过

同一条消息	结果
SHA-256(secret ‖ m)	能被续写 —— 长度扩展仍然成立
HMAC-SHA-256	不能从 tag 续写 —— 内外层都用密钥

> 正确 tag 通过：True；错误 tag 通过：False
>
> 密码学代码优先用标准库 —— **不要自己发明 MAC 构造**



实验材料

hash-lab
课件-哈希长度扩展与碰撞攻击.pdf
实验报告模板-哈希的长度扩展与碰撞攻...

code
data
tools
0-实验指导.pdf
1-实验原理.pdf
2-实验步骤.pdf

环境要求：Python ≥ 3.8，零第三方依赖。先跑 `python check_env.py`—— 全绿才能开始



提交清单

#	文件	说明
1	data/forged_url.txt	主交付物，服务端返回 200 且执行 UnlockAllSafes
2	data/good、data/evil	两个可执行文件，MD5 相同，行为不同
3	data/task1_padding.json	任务一的填充与续写数据
4	data/defense.json	任务四的对照结果
5	code/ 里填好的四个 task*.py	完整代码
6	实验报告（Word）	按模板作答 6 个问题



实验要求

- 提交内容

#	交什么
1	data/forged_url.txt（主交付物）
2	data/good、data/evil（同 MD5 程序）
3	data/task1_padding.json、data/defense.json
4	code/ 里填好的 task1~4
5	实验报告（6 个问题都要作答）

- 截止时间

下一次实验课前提交至 HITsz Grader 作业提交平台，具体截止日期参考平台发布。

- 登录网址：`https://grader.hitsz.edu.cn/`
- 推荐浏览器：Chrome

请同学们开始实验

